CSC 436 Assignment

Database Views & Normalization

Overview

In this assignment, you will enhance your group's database application by implementing SQL views for controlled data access and applying normalization techniques to achieve Third Normal Form (3NF). You will create views to restrict access to specific data and write SQL commands for managing privileges. Additionally, you will analyze your database for normalization violations, make necessary modifications to improve data integrity, and document your work with SQL queries, screenshots, and a detailed explanation of your normalization process.

Part I: SQL Views & Authorization

In this part of the assignment, you will create views in MySQL to control data access and enhance security. Views allow you to present a subset of your data to specific users without granting them full access to the underlying tables. This is particularly useful for protecting sensitive information while still providing necessary data access to different types of users.

Since the MySQL version in your cPanel does not support role-based privileges or granting user-specific permissions through phpMyAdmin, you will also write (but not execute) SQL commands that would typically be used to manage access.

A. Setup

Before creating views, complete the following:

- ☑ ~~Complete the Logging Into cPanel section to access your cPanel account.~~
- ☑ ~~Since you will be creating views, you need to verify that your database user has the necessary privileges. Complete the Updating A User's Privileges section and ensure your database user has been granted **All Privileges** in phpMyAdmin.~~
- ☑ ~~Complete the Accessing The Database section to access the database you created.~~
- ☑ ~~Review the SQL Statements section to familiarize with common SQL statements to effectively communicate with your database.~~
- ☑ ~~Review the Querying The Database section to write queries and ask questions about the data.~~

B. Creating Views

You will create at least two views that restrict access to certain data. Consider the following:

- A **limited-access view** that hides sensitive information (i.e., removing customer payment details).
- A **role-based view** that only displays records relevant to a specific user type (i.e., showing only active rental transactions for employees).

After creating your views, they will appear under the **Views** section at the bottom of your database.

C. Simulating Role-Based Access (Written SQL Only, No Execution)

Since the MySQL version in cPanel does not support granting privileges or creating roles, you will write (but not execute) the SQL commands that would typically be used to:

- Grant privileges on a view to a specific user.
- Create a role and assign users to it.

Even though these commands will not execute in your current setup, understanding them is essential for real-world database management.

Part II: Database Normalization

In this part of the assignment, you will analyze and refine your group's database application design to ensure compliance with third normal form (3NF) by identifying and addressing normalization issues.

A. Database Normalization

Normalization helps eliminate redundancy, inconsistencies, and update anomalies by organizing data efficiently. You will review your current database design and apply normalization techniques as needed.

Familiarize yourself with the following rules to analyze your database for normalization violations:

- **First Normal Form (1NF)**
    - Using row order to convey information is not permitted.
    - Mixing data types within the same column is not permitted.
    - Having a table without a primary key is not permitted.
    - Storing repeating groups or multi-valued attributes is not permitted.

- **Second Normal Form (2NF)**
    - Each non-key attribute must depend on the entire primary key.

- **Third Normal Form (3NF)**
    - Every non-key attribute in a table should depend on the key, the whole key and nothing but the key.

Identify non-conforming elements in your existing database schema. Modify tables by restructuring attributes and relationships as needed to ensure that your final design meets 3NF requirements while maintaining functionality. This may involve breaking down tables, creating new tables, and redefining relationships to eliminate redundancy while ensuring data integrity.

- *Added table Department to make the table Lab conform to 3NF, since columns department name and building name rely on each other.*

Part III: Documentation

You will compile and explain your SQL queries, screenshots, written privilege management commands, and normalization analysis.

1. Document the exact SQL queries you used to define each view. Include a brief explanation of each view's purpose and expected results.

    - In our first view we wanted to limit the access for the User data, so we created a view limiting **user_id**, **role** and **lab_id**. During that process of creating the view we selected Algorithm to be undefined, and we added a Definer user and selected SQL security to be DEFINER. We gave the view name "limited" and then this is the SQL query we wrote.

      ```
      SELECT
        first_name,
        last_name,
        email
      FROM
        `User`
      ```

- In our second view, we want to switch access a user can have based on their role and assign users to a specific role. The expected result is that user1 should be assigned to the role of Lab_manager and user2 should be assigned to the Student_Worker role. And based on their role, the users should have different levels of privileges on the tables and see different kinds of information being displayed when they access the table. We have to create two views to simulate the two versions of view that mimic what a real-world database view could look like, and also allow us to simulate two users having two different access to the Document table data. So a student worker can only see doc_type, file_name, file_url, notes of a document while a lab manager can see everything about a document. The privilege granting and role assigning SQL statements that was used is included in question 3.

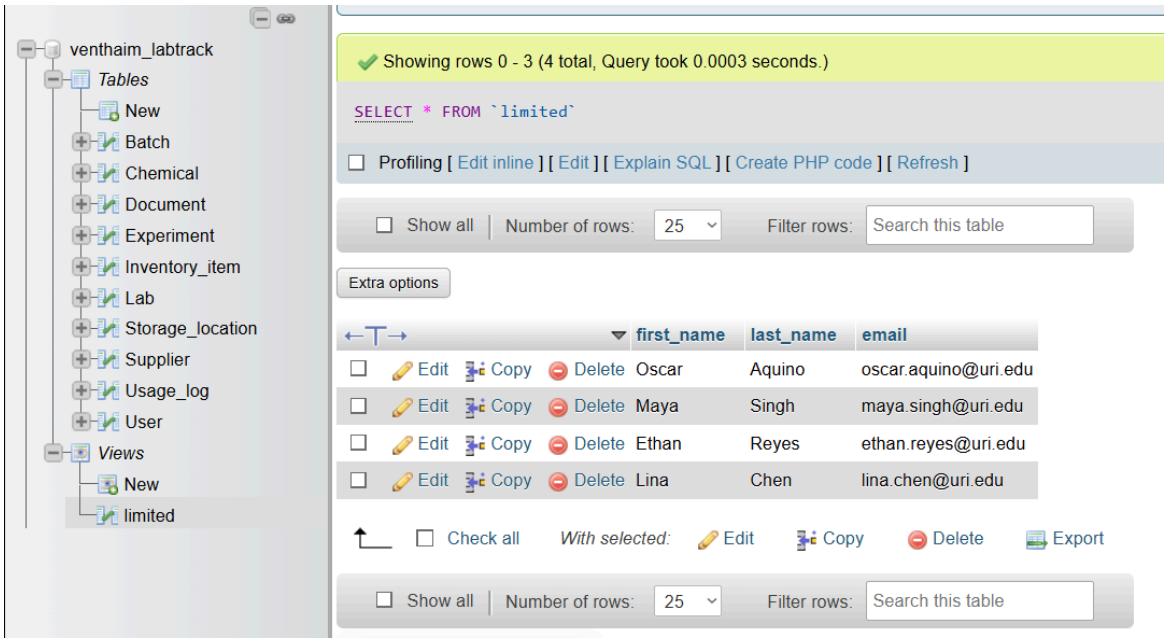
  Student worker view:

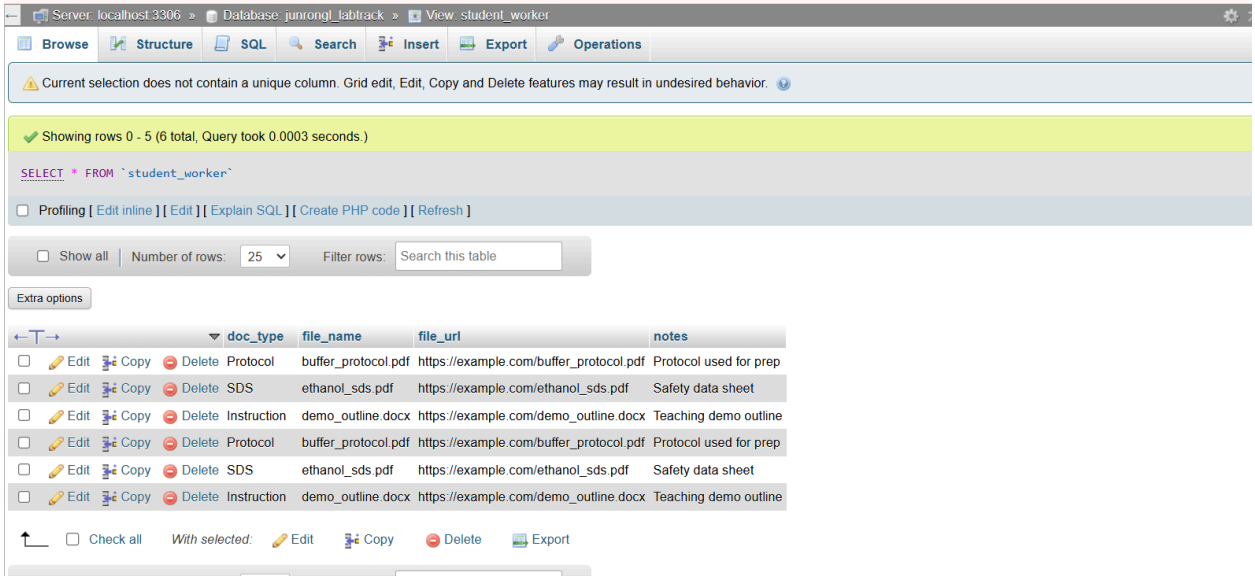
  ```
  SELECT doc_type, file_name, file_url, notes
  FROM `Document`;
  ```

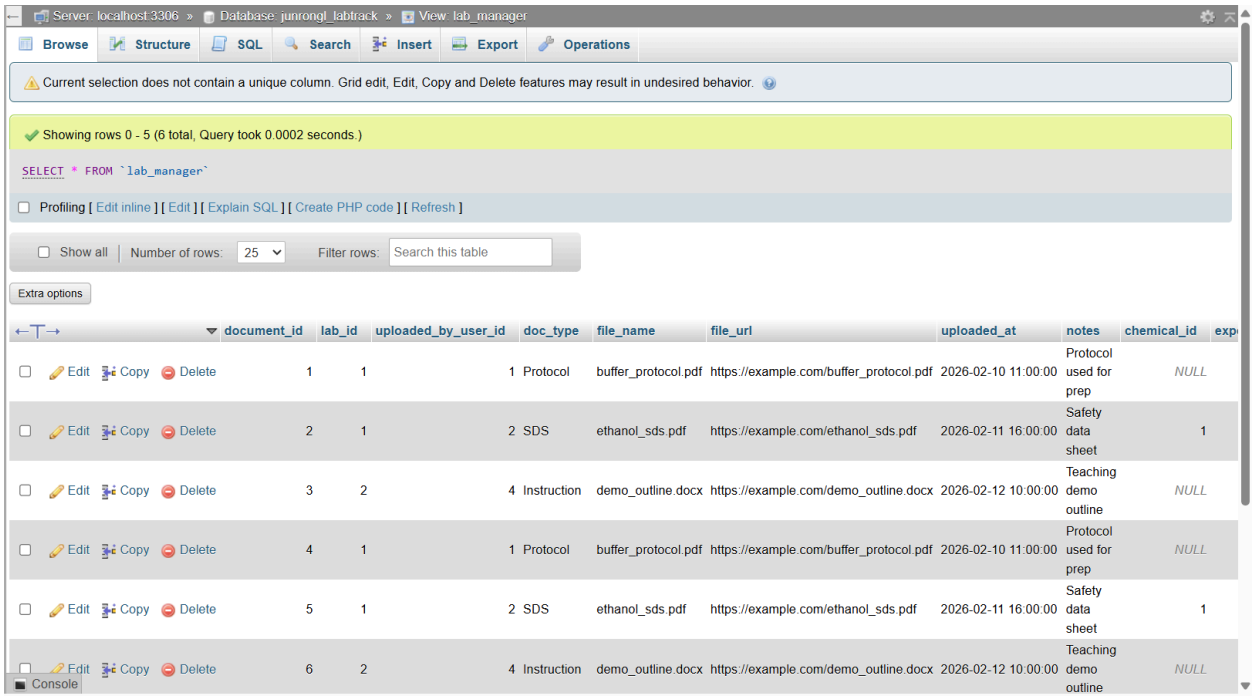
  Lab manager view:

  ```
  SELECT *
  FROM `Document`;
  ```

2. Take screenshots showing the data displayed under each view. You can use the **Browse** tab and take a screenshot of the data that is displayed under the **Views** section. This will demonstrate that your views were successfully created.




Server: localhost:3306 » Database: junrongl_labtrack » View: student_worker
Browse Structure SQL Search Insert Export Operations
Current selection does not contain a unique column. Grid edit, Edit, Copy and Delete features may result in undesired behavior.
Showing rows 0 - 5 (6 total, Query took 0.0003 seconds.)
SELECT * FROM `student_worker`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
doc_type file_name file_url notes
Edit Copy Delete Protocol buffer_protocol.pdf https://example.com/buffer_protocol.pdf Protocol used for prep
Edit Copy Delete SDS ethanol_sds.pdf https://example.com/ethanol_sds.pdf Safety data sheet
Edit Copy Delete Instruction demo_outline.docx https://example.com/demo_outline.docx Teaching demo outline
Edit Copy Delete Protocol buffer_protocol.pdf https://example.com/buffer_protocol.pdf Protocol used for prep
Edit Copy Delete SDS ethanol_sds.pdf https://example.com/ethanol_sds.pdf Safety data sheet
Edit Copy Delete Instruction demo_outline.docx https://example.com/demo_outline.docx Teaching demo outline
Check all With selected: Edit Copy Delete Export


Server: localhost:3306 » Database: junrongl_labtrack » View: lab_manager
Browse Structure SQL Search Insert Export Operations
Current selection does not contain a unique column. Grid edit, Edit, Copy and Delete features may result in undesired behavior.
Showing rows 0 - 5 (6 total, Query took 0.0002 seconds.)
SELECT * FROM `lab_manager`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
document_id lab_id uploaded_by_user_id doc_type file_name file_url uploaded_at notes chemical_id
Edit Copy Delete 1 1 1 Protocol buffer_protocol.pdf https://example.com/buffer_protocol.pdf 2026-02-10 11:00:00 Protocol used for prep NULL
Edit Copy Delete 2 1 2 SDS ethanol_sds.pdf https://example.com/ethanol_sds.pdf 2026-02-11 16:00:00 Safety data sheet 1
Edit Copy Delete 3 2 4 Instruction demo_outline.docx https://example.com/demo_outline.docx 2026-02-12 10:00:00 Teaching demo outline NULL
Edit Copy Delete 4 1 1 Protocol buffer_protocol.pdf https://example.com/buffer_protocol.pdf 2026-02-10 11:00:00 Protocol used for prep NULL
Edit Copy Delete 5 1 2 SDS ethanol_sds.pdf https://example.com/ethanol_sds.pdf 2026-02-11 16:00:00 Safety data sheet 1
Edit Copy Delete 6 2 4 Instruction demo_outline.docx https://example.com/demo_outline.docx 2026-02-12 10:00:00 Teaching demo outline NULL
Console

3. Include the SQL statements you would have executed to grant privileges on a view and create a role. Document each query's intended purpose, expected results, and actual SQL command.

```sql
CREATE ROLE 'Student_worker';
CREATE ROLE 'Lab_manager';

GRANT SELECT, UPDATE, ALTER on lab_manager TO 'Lab_manager';
GRANT SELECT, UPDATE on student_worker TO 'Student_worker';

GRANT 'Student_Worker' TO User2@localhost;
GRANT 'Lab_manager' TO User1@localhost;
```

In order to have a role based access control, we created two roles using the SQL statements called Student_worker and Lab_manager. The intended purpose of this query was to define a role based access that gives out certain privileges to the users. The expected result was that both roles would be created in the database. After creating the roles, we can assign them to specific access to the document table. For example the **Student_worker** role will have SELECT and UPDATE privilege, while the **Lab_manager** role will have SELECT, UPDATE, and ALTER privilege. The different views will also give users of different roles access to different kinds of data or columns from the table. For example, users in **Student_worker** role and in student_worker view cannot see the document ID of the documents, while a user in **Lab_manager** and lab_manager view could.

4. Identify and explain each non-conforming element in your database schema. Describe how it violates normalization principles and why it needs to be addressed.

In this schema, several elements don't fully follow third normal form (3NF), and could cause data problems over time. The same information is stored in multiple places, such as **lab_id** appearing across many tables even though lab details already depend on the **Lab** table, which can lead to inconsistencies if lab information changes. The **Usage_log** table is another issue because it stores multiple user references and a unit field that can already be determined from the inventory item or chemical, creating redundancy and making the data harder to keep consistent. The **Document** table tries to represent documents for experiments, chemicals, and labs all at once, which mixes different relationships into a single table and results in many nullable or unclear fields. There are also hidden dependencies where inventory items reference batches, even though batch records already determine chemical and supplier details, which risks update and deletion

anomalies.Overall, these issues violate normalization principles by allowing non-key attributes to depend on other non-key attributes or by duplicating the same facts in multiple tables, and they should be addressed to reduce redundancy, prevent inconsistencies, and make the database easier to maintain and query reliably.

5. Provide a detailed explanation of changes made during the normalization process. Justify each modification, explaining how it improves database efficiency, eliminates redundancy and inconsistencies, and ensures 3NF compliance.

During the normalization process, several structural changes were made to ensure the database complies with **third normal form (3NF)** and to improve consistency, efficiency, and maintainability. First, attributes that described the same real-world concept were consolidated into single tables to eliminate redundancy. For example, all lab-related details (such as lab name, building, room number, and department) were kept exclusively in the **Lab** table, while other tables retained only a **lab_id** foreign key. This change removes transitive dependencies, ensuring that non-key attributes no longer depend on other non-key attributes, and prevents update anomalies when lab information changes.

Next, the **Usage_log** table was simplified by removing redundant attributes. Multiple user identifiers were reduced to a single, clearly defined foreign key that identifies who performed the usage action. Units of measurement were also removed from the usage log, since units are already determined by the related inventory item or chemical. This ensures each fact is stored once, avoids inconsistencies when units change, and improves query efficiency by relying on joins rather than duplicated data.

The **Document** structure was normalized by separating mixed responsibilities. Instead of one table attempting to link documents to labs, experiments, and chemicals simultaneously, the design was adjusted so that documents are linked through clear, specific relationships (or junction tables where appropriate). This removes ambiguous and nullable fields, enforces clearer constraints, and aligns each table with a single purpose, which is a core principle of 3NF.

Inventory and batch data were also refined by ensuring that batch-specific attributes (such as supplier, expiration date, concentration, and lot number) exist only in the **Batch** table. The **Inventory_item** table now references the batch by ID without duplicating batch details. This eliminates partial dependencies and prevents inconsistencies if batch information needs to be updated or corrected.

Submission

When you're finished, complete the following steps to submit your work:

- ☐ Export your document with responses as a **PDF file AND save** it **inside** your **documentation** folder. Refer to the following for documentation on how to do this:
    - Google Docs (*File → Download → PDF Document*)
    - Microsoft Word (*File → Save As / Export → PDF*)
    - Pages (*File → Export To → PDF*)
- ☐ Inside your repo, export your updated database as an **SQL file** into the **db** folder. Follow the steps in the Exporting A Database section to export your database into a single SQL file.
- ☐ All group members should then be able to pull this SQL file from the repo and upload it to their own cPanel accounts (see Importing A Database section).
- ☐ Upload all your changes to GitHub.
    - ☐ **If you're using GitHub Desktop (GUI)**, complete the Uploading Changes (GitHub Desktop) section to upload your changes from your local device to GitHub.
    - ☐ **If you're using Git (CLI)**, complete the Uploading Changes (GitHub CLI) section to upload your changes from your local device to GitHub.

ONE group member must paste the URL of your GitHub repository in the provided textbox in Brightspace. Click the blue *Submit* button to successfully submit your work for this assignment.

Grading Rubric

You can refer to the **Database Views & Normalization grading rubric** given in Brightspace for this assignment to find details on how your submission will be graded.